b. Investigación

- **DevOps:**

¿Qué significa DevOps, para que sirve y cómo funciona? ¿Qué relación tiene con las metodologías Ágiles y cuales son sus diferencias? Realizar un gráfico de DevOps mostrando las herramientas que se sugieren utilizar explicando cada una de ellas.

DevOps es la combinación de Development (Desarrollo) y Operations (Operaciones). No es una herramienta ni un software, sino una cultura, filosofía y conjunto de prácticas que busca romper las barreras entre los programadores (quienes crean el software) y los administradores de sistemas (quienes mantienen los servidores).
Sirve para acelerar el ciclo de vida del desarrollo de software.

Funciona a través de un flujo de trabajo continuo que abarca todo el ciclo de vida de la aplicación. Se basa en la automatización mediante metodologías CI/CD (Integración Continua y Despliegue Continuo). La idea es que desde que un desarrollador termina de escribir una línea de código, esta pase por sistemas automáticos de revisión, pruebas de calidad y empaquetado, hasta llegar a los servidores de producción con la mínima intervención humana posible.

DevOps es como la extensión o evolución de las metodologías ágiles. Mientras que las metodologías ágiles optimizan la forma en que los equipos planifican el trabajo y escriben el software en ciclos cortos, DevOps toma ese software y garantiza que también se pruebe, se despliegue y se monitoree de forma rápida y continua.

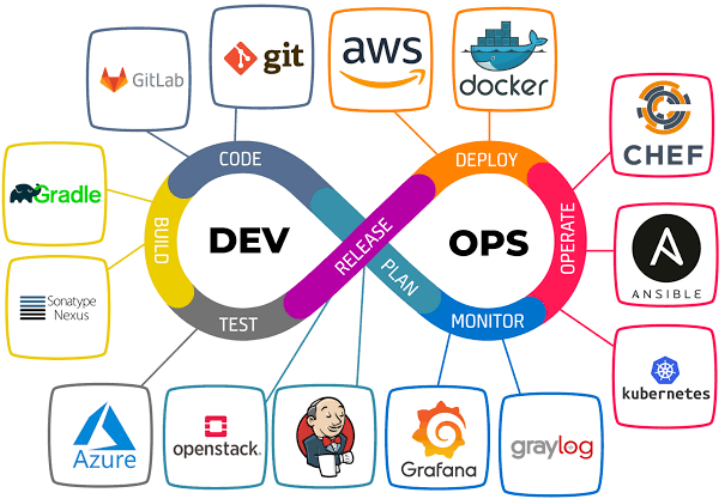
Diferencias:

- Ágil se enfoca en el proceso de desarrollo y DevOps abarca todo el ciclo completo, extendiéndose hasta la infraestructura y el mantenimiento.
- Ágil se apoya mucho en procesos, reuniones y roles humanos. DevOps se apoya mucho en la ingeniería y en herramientas tecnológicas de automatización.
- En ágil el éxito suele medirse al final de un sprint. En DevOps los despliegues a producción pueden ocurrir muchas veces en un mismo día.

Herramientas en cada fase:

- Git / GitHub / GitLab (Fase de Código): Son sistemas de control de versiones. Permiten que múltiples desarrolladores trabajen en el mismo proyecto al mismo tiempo sin sobrescribir el trabajo del otro.
- Grafana y Graylog (Monitoreo): Grafana crea tableros interactivos para visualizar métricas de rendimiento en tiempo real, mientras que Graylog centraliza el análisis de registros (logs) para detectar errores del sistema.
- Chef, Ansible y Kubernetes (Operación): Chef y Ansible son herramientas que automatizan la configuración masiva de los servidores mediante código, evitando instalaciones manuales. Kubernetes es un orquestador que administra miles de contenedores simultáneamente, escalando recursos si hay mucho tráfico y reiniciando servicios caídos.
- Docker (Despliegue): Empaqueta la aplicación y sus dependencias en contenedores aislados para garantizar que funcionen de manera idéntica en cualquier servidor.

- Jenkins (Lanzamiento): Motor principal de automatización (CI/CD) que orquesta todo el proceso desde que el programador sube el código hasta su preparación para el despliegue.
- AWS, Azure y OpenStack (Infraestructura en la Nube): Proveedores de recursos de computación (públicos en el caso de AWS y Azure, privados en OpenStack) para alojar la aplicación y sus bases de datos.
- Gradle y Sonatype Nexus (Construcción): Gradle es una herramienta que automatiza la compilación, la ejecución de pruebas y el empaquetado del software. Sonatype Nexus es un repositorio que almacena y administra de forma segura los binarios, contenedores y librerías generadas en el proyecto.



- **Ambiente de Producción y Despliegue en Producción:**

¿Qué significa que una aplicación se despliega en un entorno de Producción? ¿Cuál es la diferencia en el desarrollo de software de un entorno de desarrollo, entorno de pruebas y entorno de producción?

Desplegar (*deploy*) es publicar una aplicación en un servidor o infraestructura donde pueda ser usada. El **entorno de producción** es el ambiente real y en vivo donde la aplicación corre para los **usuarios finales**. Allí hay datos reales, tráfico real y consecuencias reales: si algo falla, los usuarios lo notan, se pierden ventas o se exponen datos de personas.

Por eso el código no pasa directo del programador a producción. Antes recorre otros entornos, cada uno con un propósito distinto, para detectar errores donde salen baratos y no donde salen caros.

Entorno de desarrollo (Development)

- Es donde los programadores escriben y prueban el código a medida que trabajan, generalmente en su propia computadora o en un servidor compartido del equipo.
- Cambia constantemente y puede romperse sin que pase nada grave.
- Usa datos falsos o de ejemplo, con herramientas de depuración activadas y mensajes de error detallados.

- Tiene pocos recursos y poca seguridad, porque no hace falta más.

Entorno de pruebas (Testing / QA)

- Es donde se verifica que lo desarrollado funciona bien: pruebas funcionales, de integración, de regresión y de aceptación, tanto automáticas como manuales, a cargo del equipo de QA.
- Es más estable que desarrollo, porque recibe versiones ya terminadas para evaluar.
- Usa datos de prueba controlados y repetibles.
- Muchas empresas agregan además un entorno de **staging** (preproducción): una copia casi idéntica a producción, para el ensayo final antes de publicar.

Entorno de producción (Production)

- Es el que usan los clientes reales.
- Debe ser estable, rápido, seguro y estar disponible casi siempre.
- Maneja datos reales y sensibles, por lo que requiere cifrado, copias de seguridad, control de accesos y cumplimiento de normativas.
- Tiene monitoreo y alertas permanentes, y los cambios se hacen de forma controlada y con plan de reversión (*rollback*).
- Los errores no se muestran en detalle al usuario, sino que se registran en logs.

¿Qué es un dominio de internet, para qué sirve y cómo funciona? Dar 5 ejemplos de dominios conocidos¿Qué es un servidor DNS (Sistema de Nombre de Dominio)? ¿Cómo se pueden obtener un dominio en Argentina y en el resto del mundo? ¿Cómo puede obtenerse un dominio gratuito?

Un **dominio de internet** es una cadena alfanumérica única que sirve como dirección identificadora de un sitio web, servidor o servicio en la red.

- Su objetivo principal es sustituir las direcciones IP numéricas complejas (como 142.250.190.46 en IPv4 o direcciones IPv6) por nombres fáciles de recordar, facilitando el acceso a sitios, servicios de correo electrónico y servidores web.
- Cuando ingresas un dominio en el navegador, este no puede comunicarse directamente con el servidor usando letras; necesita la dirección IP. El navegador inicia una consulta a través de la infraestructura de red hacia el sistema DNS, el cual traduce ese nombre en la dirección IP del servidor donde está alojado el contenido, permitiendo establecer la conexión TCP/IP correspondiente.

Algunos dominios conocidos son:

- google.com
- wikipedia.org
- github.com
- mercadolibre.com.ar
- uba.ar

Un **servidor DNS** (*Domain Name System*) es un servidor especializado que mantiene bases de datos distribuidas y jerárquicas con registros (A, AAAA, CNAME, MX, etc.) para traducir nombres de dominio legibles en direcciones IP numéricas que las computadoras y enrutadores comprenden. Cuando un usuario escribe un dominio, el resolvedor DNS consulta recursivamente a los servidores raíz, a los servidores TLD (*Top-Level Domain*) y finalmente al servidor autoritativo del dominio para obtener la IP exacta.

¿Cómo se puede obtener un dominio en Argentina y en el resto del mundo?

- **En Argentina:**
    1. Los dominios con terminación .ar (como .com.ar, .ar, .net.ar) están gestionados por el organismo estatal **NIC Argentina** (dependiente de la Secretaría Legal y Técnica).
    2. Para registrar uno, se debe ingresar al sitio oficial nic.ar utilizando la plataforma **TAD** (Trámites a Distancia) con CUIT/CUIL y Clave Fiscal de AFIP.
    3. Se busca la disponibilidad del nombre, se completan los datos del titular y se abona el arancel anual correspondiente.
- **En el resto del mundo (dominios globales como .com, .net, .org):**
    1. Están regulados globalmente por la **ICANN** (*Internet Corporation for Assigned Names and Numbers*).
    2. Se adquieren a través de empresas registradoras acreditadas (*Registrars*) como Namecheap, GoDaddy, Google Domains / Squarespace, Cloudflare o DonWeb.
    3. El usuario crea una cuenta, verifica la disponibilidad del dominio, abona la tarifa periódica y configura los servidores de nombres (*Name Servers* o NS) para apuntar a su hosting o proveedor DNS.

¿Cómo puede obtenerse un dominio gratuito?

- **Subdominios gratuitos de plataformas de hosting:** Servicios como GitHub Pages (usuario.github.io), Vercel (proyecto.vercel.app), Netlify (sitio.netlify.app) o Firebase Hosting (proyecto.web.app) brindan un subdominio gratuito al desplegar proyectos web.
- **Proveedores de DNS dinámico (DDNS):** Plataformas como No-IP, DuckDNS o FreeDNS ofrecen nombres de dominio/subdominios gratuitos (ej. mi-servidor.duckdns.org), comúnmente utilizados para pruebas locales o servidores caseros.
- **Programas educativos:** Iniciativas como el *GitHub Student Developer Pack* proporcionan a estudiantes registros gratuitos por un año de dominios .me o .tech mediante convenios con registradores comerciales.

¿Qué es un servidor web? ¿Qué significa poder hacer Hosting de una aplicación?
¿Qué características y costos tiene DON WEB para aplicaciones de tipo SpringBoot?
¿Qué servidores web internacionales existen para realizar Hosting?

Un servidor web es el conjunto de un programa de software y un equipo informático que almacenan y entregan el contenido de un sitio web a través de Internet a los navegadores de los usuarios. Poder hacer hosting de una aplicación significa alojar y publicar ese

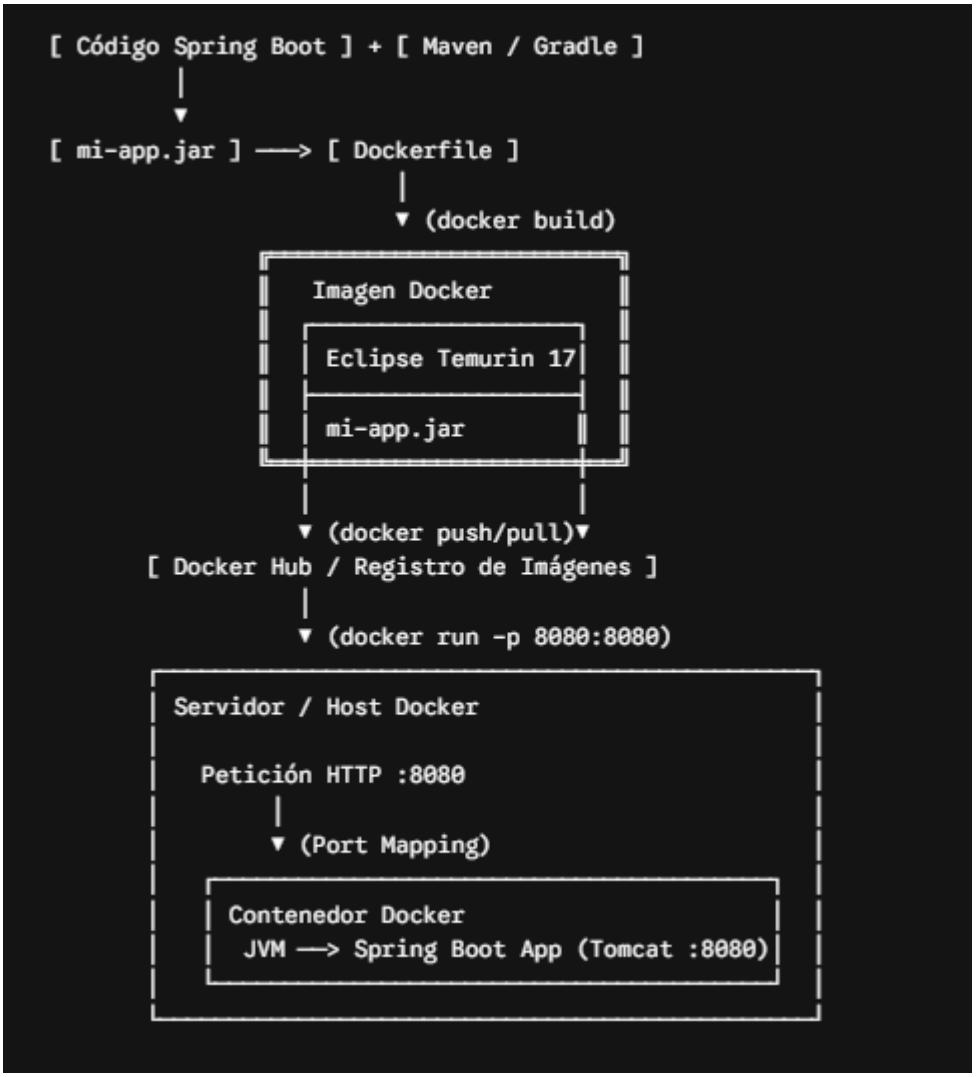
software en un servidor remoto de alta disponibilidad (en lugar de una computadora personal), permitiendo que usuarios de cualquier parte del mundo puedan acceder a él a través de Internet de forma ininterrumpida.

Servidores web internacionales para realizar Hosting:

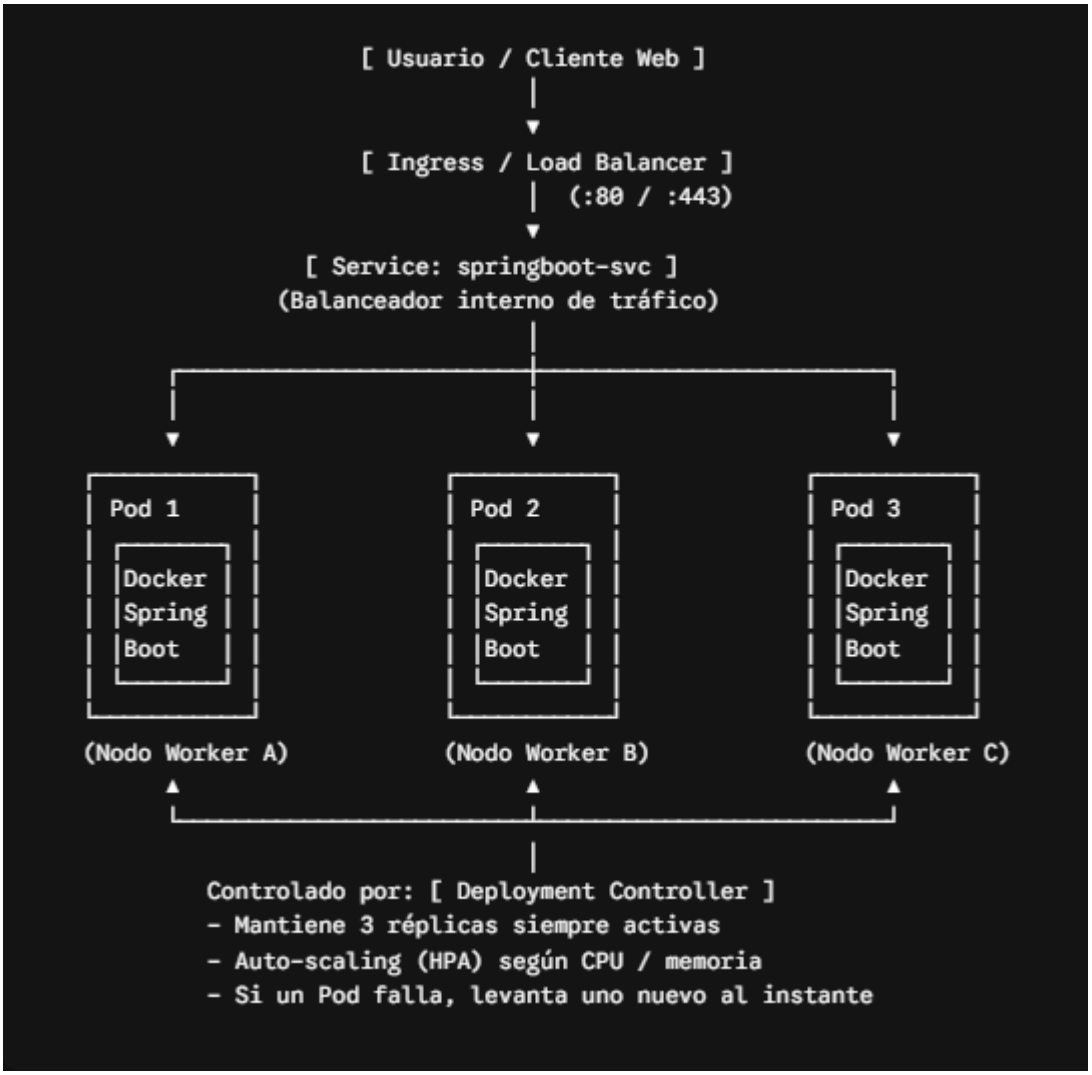
- Amazon Web Services (AWS): Infraestructura en la nube líder que permite un alojamiento escalable.
- GitHub Pages: Servicio de alojamiento estático directamente desde un repositorio.
- Vercel y Netlify: Plataformas orientadas al frontend y despliegue rápido.
- Render: Proveedor de tipo PaaS (Plataforma como Servicio)

¿Qué es DOCKER y para qué sirve? Realizar de manera Gráfica de funcionamiento para una aplicación SpringBoot. ¿Qué es KUBERNET y para qué sirve? Realizar de manera Gráfica de funcionamiento para una aplicación SpringBoot. Mencionar las diferencias entre DOCKER y KUBERNET.

Docker es una plataforma de código abierto basada en contenedores que permite empaquetar una aplicación y todas sus dependencias en una unidad estandarizada llamada **contenedor**. Sirve para garantizar que el software se ejecute de manera idéntica y predecible en cualquier entorno (desarrollo, pruebas o producción), aislando procesos y optimizando el uso de recursos frente a las máquinas virtuales tradicionales.



Kubernetes (a menudo abreviado como **K8s**) es una plataforma de orquestación de contenedores de código abierto. Sirve para automatizar el ciclo de vida completo de aplicaciones contenerizadas a gran escala: despliegue, balanceo de carga, escalabilidad horizontal automática, reinicio automático en caso de fallos (*self-healing*) y actualizaciones progresivas sin tiempo de inactividad (*rolling updates*). Mientras Docker crea y ejecuta los contenedores de forma aislada, Kubernetes administra múltiples contenedores distribuidos a lo largo de un clúster de servidores.



Criterio	Docker	Kubernetes (K8s)
Rol principal	**Contenerización:** Crea, empaqueta y ejecuta contenedores individuales.	**Orquestación:** Administra, coordina y escala cientos de contenedores en red.
Alcance	Funciona habitualmente en una sola máquina o nodo (servidor local).	Opera sobre un clúster distribuido de múltiples servidores físicos o virtuales.

Escalabilidad	Manual (o básica con Docker Compose / Swarm).	Automática y dinámica en base a consumo de CPU, memoria o métricas personalizadas.
Tolerancia a fallos (*Self-healing*)	Si el contenedor muere, requiere scripts o reinicio manual salvo políticas básicas.	Supervisa el estado de salud y recrea pods defectuosos automáticamente de inmediato.
Balanceo de carga y red	Mapeo de puertos directo entre el host y el contenedor.	Dispone de balanceadores de carga internos y externos integrados (*Services* e *Ingress*).
Relación mutua	Son complementarios: Docker construye y empaqueta la imagen de Spring Boot que Kubernetes ejecuta y gestiona en sus Pods.	

- **Integración Continua y Despliegue Continuo:**

¿Qué significa Integración Continua (CI) y que significa despliegue continuo (CD)?, ¿Cuáles son sus ventajas? Realizar de manera Gráfica el proceso de CI/CD en el desarrollo de una aplicación.

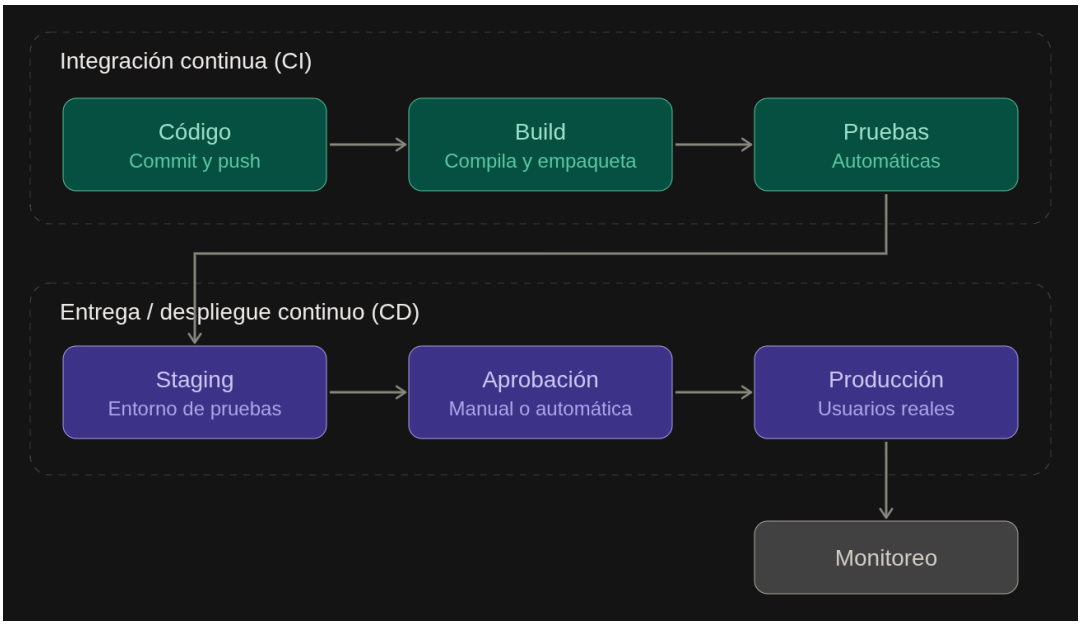
Qué es CI (Integración Continua)

Es la práctica de que cada desarrollador **integre su código en el repositorio compartido con mucha frecuencia** (varias veces al día). Cada vez que se sube un cambio, un sistema automático lo **compila y ejecuta las pruebas**. Así los errores de integración se detectan en minutos, y no semanas después, cuando arreglarlos es mucho más caro.

Qué es CD (Entrega / Despliegue Continuo)

La sigla CD puede significar dos cosas parecidas:

- **Entrega continua (*Continuous Delivery*):** el código que pasó las pruebas queda siempre **listo para publicarse**, pero el paso final a producción requiere una **aprobación manual**.
- **Despliegue continuo (*Continuous Deployment*):** va un paso más allá. Si todo pasa las pruebas, se **publica en producción automáticamente**, sin intervención humana.



Ventajas

De la integración continua:

- Detecta errores temprano, cuando son más baratos y fáciles de corregir.
- Evita los conflictos grandes y dolorosos al unir el trabajo de varios programadores.
- Mantiene el código siempre en un estado funcional.
- Mejora la calidad, porque las pruebas se ejecutan siempre y de forma consistente.

De la entrega y el despliegue continuo:

- Lanzamientos más rápidos y frecuentes, con menor tiempo entre una idea y su uso real.
- Menos riesgo, porque los cambios pequeños son más fáciles de probar y revertir que los grandes.
- Elimina tareas manuales repetitivas y propensas a errores humanos.
- Feedback más rápido de los usuarios, lo que permite corregir el rumbo antes.
- Despliegues repetibles y predecibles.
- El equipo se concentra en desarrollar, no en publicar.